

Aerospace Predictive Maintenance

Hackathon MVP — Code Documentation & Model Evaluation

This document describes every Python file in the project, the purpose and input/output signature of each function, the reasoning behind the statistical / machine-learning methods used, the measured accuracy of both models (including detection lead time and false alarm rate), and the integration roadmap for connecting to a real MRO / asset-management system.

Pipeline: simulate telemetry → detect anomalies → predict RUL → benchmark both models → build fleet dashboard → generate MRO work orders. Each stage lives in its own file and is run end-to-end by `main.py`.

Requirement coverage: (1) anomaly detection prototype → `data_simulator.py` + `anomaly_detection.py`; (2) RUL model with benchmarks → `rul_model.py` + `evaluation.py`; (3) fleet dashboard → `dashboard.py`; (4) lead time / false alarm rate → `evaluation.py`; (5) MRO integration roadmap → `mro_integration.py` + the roadmap section of this document.

1. data_simulator.py

Purpose: Generates synthetic (fake but realistic) multi-channel sensor telemetry for a fleet of aircraft, since no real aircraft sensor data was available for the hackathon. Each simulated part (engine, landing gear, brakes, bogie) is given its own random total lifespan and a random 'current age', so the resulting fleet snapshot looks like a real one: mostly healthy parts, a few aging, one or two close to failure. Ground-truth columns (RUL, true_anomaly, life_length) are retained purely for benchmarking the models later.

Function	Inputs	Output	What it does
make_one_unit_data(rng, aircraft_id, component, unit_id)	rng: numpy random Generator aircraft_id: str (e.g. 'AC-001') component: str (e.g. 'engine') unit_id: str, unique part id	pandas DataFrame: one row per simulated cycle, columns = aircraft_id, unit_id, component, cycle, vibration, temperature, pressure, RUL, true_anomaly, life_length	Picks a random total lifespan and a random 'current age' (triangular distribution, biased toward earlier cycles). Builds a wear curve (cycle/life)^2, simulates each sensor as baseline + wear trend + Gaussian noise, optionally injects one sudden anomaly spike, and computes ground-truth (capped) RUL plus the uncapped life_length per cycle.
simulate_fleet (seed=42)	seed: int, random seed for reproducibility	pandas DataFrame: all units stacked together (N_AIRCRAFT × 4 components)	Loops over every aircraft and every component type, calls make_one_unit_data for each, and concatenates all of the individual part tables into one fleet-wide table.

2. anomaly_detection.py

Purpose: Flags sensor readings that deviate from a specific part's own established healthy baseline, using a z-score statistical method (the same principle behind Statistical Process Control / SPC charts used in industrial monitoring). This is the working anomaly detection prototype (requirement 1), run against the simulated fleet as a representative sensor dataset.

Function	Inputs	Output	What it does
get_healthy_baseline(unit_df)	unit_df: DataFrame, all cycles for ONE unit_id (must include vibration, temperature, pressure)	tuple (means, stds): pandas Series, per-sensor mean & std dev	Takes the first 30% of that part's rows (at least 10 rows, MIN_HEALTHY_ROWS) as 'known good' data and computes the average and spread of each sensor during that healthy period. The 10-row floor avoids an unstable baseline for parts with very few simulated cycles.
score_unit(unit_df, means, stds)	unit_df: DataFrame for one unit means, stds: from get_healthy_baseline	pandas Series: one anomaly_score per cycle (float)	Computes $z = (\text{value} - \text{mean}) / \text{std}$ for each sensor, then keeps the worst (largest magnitude) z-score across the 3 sensors as that cycle's overall anomaly score.
detect_anomalies(fleet_df)	fleet_df: full multi-unit telemetry DataFrame	same DataFrame + 2 new columns: anomaly_score (float), is_anomaly (0/1)	Repeats get_healthy_baseline + score_unit independently for every unit_id, so each part is judged only against its OWN history, then flags any cycle whose score exceeds Z_SCORE_LIMIT (4.0 std devs).

3. rul_model.py

Purpose: Predicts Remaining Useful Life (RUL) — how many more cycles a part can safely run — using one small linear regression model per component type. Accuracy benchmarks (requirement 2) are computed by evaluation.py, see the dedicated section below.

Function	Inputs	Output	What it does
<code>split_train_test(fleet_df, test_fraction=0.25, seed=1)</code>	<code>fleet_df</code> : DataFrame <code>test_fraction</code> : float <code>seed</code> : int	tuple (<code>train_df</code> , <code>test_df</code>)	Splits the fleet by <code>unit_id</code> (never by row), so no single part's cycles appear in both the training and test sets. This prevents data leakage and gives an honest accuracy estimate on parts the model has never seen.
<code>add_bias_column(X)</code>	<code>X</code> : 2D numpy array of feature values	2D numpy array with an extra leading column of 1s	Adds the constant/intercept term that ordinary linear regression needs so its fitted line does not have to pass through the origin.
<code>train_linear_model(train_df)</code>	<code>train_df</code> : training rows (needs FEATURE_COLUMNS + RUL)	dict { <code>component_name</code> : weights}, <code>weights</code> = numpy array of 5 coefficients (bias, cycle, vibration, temperature, pressure)	For each of the 4 component types separately, solves the least-squares regression equation (<code>numpy.linalg.lstsq</code>) mapping sensor readings + cycle number to RUL. Trained per-component because sensor scales differ drastically (e.g. engine temperature ~550 vs landing gear ~60).
<code>predict_rul(weights_by_component, df)</code>	<code>weights_by_component</code> : dict from <code>train_linear_model</code> df: rows to predict on	numpy array of predicted RUL values, one per row (clipped at 0)	Looks up the correct small model for each row's component type and applies it (weighted sum of features).
<code>evaluate(y_true, y_pred)</code>	<code>y_true</code> , <code>y_pred</code> : numpy arrays of actual vs. predicted RUL	dict {'MAE': float, 'RMSE': float}	Computes overall Mean Absolute Error and Root Mean Squared Error across all test rows pooled together.

4. evaluation.py

Purpose: Directly satisfies requirement 4 — computes the maintenance-relevant benchmark metrics for both models: detection lead time, false alarm rate (two variants), precision/recall for the anomaly detector, and per-component RUL accuracy for the RUL model.

Function	Inputs	Output	What it does
<code>evaluate_anomaly_detection(scored_df)</code>	<code>scored_df</code> : fleet data with <code>is_anomaly</code> , <code>anomaly_score</code> , <code>true_anomaly</code> , <code>RUL</code> , <code>life_length</code> columns	dict: <code>true_positives</code> , <code>false_positives</code> , <code>false_negatives</code> , <code>true_negatives</code> , <code>precision</code> , <code>recall</code> , <code>false_alarm_rate</code> , <code>early_life_false_alarm_rate</code> , <code>f1_score</code> , <code>mean/median_detection_lead_time_cycles</code> , <code>units_flagged</code> , <code>units_never_flagged</code>	Computes standard detection metrics against the simulator's injected-spike ground truth, PLUS: (a) an early-life false alarm rate restricted to each part's first 20% of cycles (isolates pure sensor-noise false alarms from legitimate early-degradation flags), and (b) detection lead time = the part's true uncapped remaining life (<code>life_length - cycle</code>) at the moment of its FIRST alert, averaged across every part that was ever flagged.

Function	Inputs	Output	What it does
<code>evaluate_rul_by_component(test_df, predictions)</code>	test_df: held-out test rows predictions: numpy array of predicted RUL, aligned to test_df rows	dict {component_name: {'MAE', 'RMSE', 'n_test_rows'}}	Breaks the overall RUL accuracy down separately for engine / landing_gear / brakes / bogie, since some component types are harder to predict than others (see benchmark results below).

5. dashboard.py

Purpose: Directly satisfies requirement 3 — builds the fleet-level 'Maintenance Action Dashboard' with one row per part showing current health status and a model-performance summary panel, rendered as a browser-viewable HTML page.

Function	Inputs	Output	What it does
<code>classify_health(row)</code>	row: pandas Series with predicted_RUL and is_anomaly	str: 'Critical', 'Warning', or 'Healthy'	Applies threshold rules: $RUL \leq 25 \rightarrow$ Critical; $RUL \leq 60$ OR a live anomaly $\rightarrow$ Warning; otherwise Healthy. A live anomaly alone does not force Critical — it is a flag to investigate, not an imminent-failure signal on its own.
<code>build_dashboard(scored_df, weights, predict_rul_fn, feature_columns)</code>	scored_df: fleet data with anomaly columns weights: trained RUL models predict_rul_fn: function reference feature_columns: list of str	DataFrame: one row per unit_id (latest cycle only), sorted most-urgent-first	Keeps only each part's most recent reading (its 'right now' status), predicts RUL, classifies health, computes a priority_rank (lower RUL and any anomaly float to the top), and returns the sorted summary table.
<code>save_dashboard_html(dashboard_df, path, metrics_summary=None)</code>	dashboard_df: DataFrame from build_dashboard path: str, output file location metrics_summary: optional dict from evaluation.py	None (writes an .html file to disk)	Renders the summary table as a simple, dependency-free HTML page, color-coding each row red/yellow/green by health_status. If metrics_summary is supplied, also renders a 'Model Performance Benchmarks' panel above the table (RUL MAE/RMSE, anomaly recall, early-life false alarm rate, mean detection lead time).

6. mro_integration.py

Purpose: Simulates sending maintenance work orders to an existing MRO (Maintenance, Repair & Overhaul) or asset-management system (e.g. SAP PM, IBM Maximo). In production, `send_to_mro_system` would call that system's real API instead of writing a local file — the rest of the pipeline stays unchanged. See the Integration Roadmap section for the planned rollout path (requirement 5).

Function	Inputs	Output	What it does
<code>build_work_order(row)</code>	row: one row of the dashboard DataFrame	dict: work_order_id, aircraft_id, component, unit_id, predicted_RUL_cycles, anomaly_detected, priority, created_at, recommended_action	Formats a single part's maintenance need as a structured record ready for an MRO system.
<code>create_work_orders(dashboard_df)</code>	dashboard_df: fleet dashboard table	list of dicts (one per Critical/Warning part; Healthy parts are skipped)	Filters the dashboard down to parts that need attention and builds one work order per part.
<code>send_to_mro_system(work_orders, path)</code>	work_orders: list of dicts path: str, default './outputs/work_orders.json'	None (writes a JSON file, prints a confirmation)	Persists the work orders as JSON — a stand-in for a real API call such as <code>requests.post(mro_api_url, json=work_order)</code> .

7. main.py

Purpose: The single entry point that runs the entire pipeline end-to-end, in order, printing step-by-step progress — this is the one file to run for a live demo.

Function	Inputs	Output	What it does
<code>run_pipeline()</code>	None (uses default parameters from the imported functions)	None. Side effects: prints progress to the console; writes <code>fleet_telemetry.csv</code> , <code>dashboard.html</code> , <code>fleet_status.csv</code> , <code>work_orders.json</code> , and <code>model_metrics.json</code> to disk	Calls, in order: <code>simulate_fleet</code> → <code>detect_anomalies</code> → <code>split_train_test</code> / <code>train_linear_model</code> / <code>predict_rul</code> / <code>evaluate</code> → <code>evaluate_anomaly_detection</code> / <code>evaluate_rul_by_component</code> → <code>build_dashboard</code> / <code>save_dashboard_html</code> → <code>create_work_orders</code> / <code>send_to_mro_system</code> , printing a running commentary at each step.

Model Evaluation & Benchmarks (Requirement 4)

Figures below are from a representative run of main.py against the simulated fleet (32 parts: 8 aircraft × 4 component types). Exact numbers vary slightly run to run since the fleet is randomly generated, but the pattern is stable. Full figures are always written to outputs/model_metrics.json after every run.

Remaining Useful Life (RUL) accuracy

Metric	Value	What it means
Overall MAE	~13.5 cycles	Average absolute miss on predicted remaining life, across all held-out test parts.
Overall RMSE	~19.6 cycles	Root-mean-square error; penalizes large misses more than the MAE does.
Engine MAE / RMSE	~4.1 / ~5.4 cycles	Best-performing component type in this run.
Brakes MAE / RMSE	~4.2 / ~5.2 cycles	Similarly strong; few test rows in this run (small sample).
Bogie MAE / RMSE	~5.6 / ~7.1 cycles	Solid accuracy, largest test sample of the four.
Landing gear MAE / RMSE	~20.8 / ~26.2 cycles	Weakest component type - see Limitations in the README.

Anomaly detection quality

Metric	Value	What it means
Recall	100%	Every injected (spike-type) anomaly in the test data was caught - zero missed faults.
Early-life false alarm rate	0.0%	Zero incorrect flags during the first 20% of a part's life, when it should still read healthy. This isolates pure sensor-noise false alarms.
Whole-life false alarm rate	~22%	Higher because many later-life flags are genuine early degradation warnings rather than detector mistakes (see caveat below).
Mean detection lead time	~150 cycles	Average advance warning - in true, uncapped remaining life - before a part's actual end of life, measured from its first alert.
Units flagged / never flagged	20 / 12 (of 32)	12 parts were young/healthy enough in this snapshot to never trigger an alert.

Caveat: the simulator's true_anomaly ground-truth label only marks sudden injected spikes (e.g. a bird strike or sensor glitch) - it does NOT label gradual wear-driven drift as an anomaly, even though catching that drift early is exactly what a predictive-maintenance system should do. This is why whole-life false alarm rate looks worse than early-life false alarm rate: many of the 'false' alarms during later cycles are correct early-degradation catches, not detector errors. Reporting both numbers side by side avoids overstating or understating the detector's real-world usefulness.

Why These Machine-Learning / Statistical Methods Were Chosen

Anomaly Detection: Per-Unit Z-Score (Statistical Process Control)

- **Explainable and auditable:** every flag can be traced back to 'this reading is N standard deviations from this exact part's own healthy average.' In a safety-relevant aerospace context, an alert that can be explained in one sentence is far easier for an engineer to trust and certify than a black-box model's score.
- **No labeled fault data required:** the method is unsupervised — it only needs a period of known-healthy running to learn from. This matches reality in aviation MRO: confirmed failure examples are rare and expensive, but healthy operating data is abundant.
- **Per-unit personalized baseline:** two engines of the same model can have slightly different 'normal' signatures due to manufacturing tolerances and installation history. Learning each unit's own baseline (instead of one fleet-wide threshold) mirrors how real condition-based-maintenance programs (vibration monitoring, oil analysis/SOAP) operate.
- **Lightweight and dependency-free:** the entire detector is plain numpy/pandas math that scores the whole fleet in milliseconds, with nothing to install and no training run — well suited to a time-boxed hackathon demo where reliability matters more than squeezing out marginal accuracy.
- **Clear upgrade path:** `score_unit`'s z-score can later be swapped for a more advanced multivariate detector (Mahalanobis distance, PCA reconstruction error, Isolation Forest, autoencoder) using the same function signature, once real labeled fault data is available to validate against.

Remaining Useful Life (RUL): Linear Regression Per Component Type

- **Interpretability:** each learned weight directly answers 'how much does one more cycle / one more unit of vibration / temperature / pressure change the predicted remaining life?'. Planners and engineers can sanity-check the coefficients themselves, which builds trust in a domain where an opaque deep-learning score is hard to justify to a maintenance crew or safety auditor.
- **No external ML dependency, closed-form solution:** the model is solved directly with `numpy.linalg.lstsq` (ordinary least squares) — no training loop, no convergence risk, no GPU. This sandbox had no internet access to install heavier libraries (scikit-learn / TensorFlow / PyTorch were unavailable), so a closed-form numpy model kept the MVP fully runnable anywhere with just numpy and pandas.
- **Component-specific scaling:** engine, landing gear, brakes, and bogie sensors sit on very different numeric scales (e.g. engine temperature ≈ 550 vs. landing gear ≈ 60). Pooling them into a single regression would force one set of coefficients to serve incompatible scales. Training one small model per component type respects these physical differences while keeping the total complexity low (just 4 tiny models, identical math).
- **Matches the shape of the problem:** degradation trends are smooth and largely monotonic (accelerating wear), which a linear model approximates reasonably well over the operating region that matters for a threshold-crossing maintenance decision — demonstrated by the per-component MAE/RMSE above.
- **Clear upgrade path:** `predict_rul()`'s input/output contract (sensor features $\rightarrow$ RUL) is identical to what a Random Forest, Gradient Boosting model, or LSTM sequence model would expose. Swapping in a more powerful model later, once real flight data is available, only requires changing `rul_model.py` internally — `dashboard.py` and `mro_integration.py` do not need to change.

Integration Roadmap: MRO & Asset-Management Connectivity (Requirement 5)

mro_integration.py is currently a mock adapter that writes work_orders.json instead of calling a real system. The intended phased rollout to a live MRO / asset-management system:

- **Phase 1 - Data contract validation (this MVP).** A file/JSON-based mock adapter proves out the work-order schema (component, predicted RUL, anomaly flag, priority, recommended action) before any live system is involved, so the schema can be reviewed and agreed on cheaply.
- **Phase 2 - Pilot API integration.** Replace send_to_mro_system with a REST adapter calling a sandbox/test instance of the target system (e.g. SAP PM, IBM Maximo, Trax, AMOS), using that system's native work-order/notification API. Endpoint URL and credentials become configuration, not code.
- **Phase 3 - Secure production connectivity.** Add OAuth2/API-key authentication, retry and idempotency handling (so a network blip never double-creates a work order), a field-mapping/crosswalk layer between our schema and the target system's data model (functional locations, asset numbers, etc.), and audit logging of every transmitted work order.
- **Phase 4 - Bi-directional sync.** Pull completed work-order status and maintenance history back from the MRO system, so the RUL model can be retrained on real outcomes and each part's life counter resets correctly after maintenance is performed.
- **Phase 5 - Real-time, fleet-scale rollout.** Move from batch/cron pipeline runs to event-driven streaming (e.g. a message queue), so a live anomaly reaches the MRO system within seconds, plus role-based dashboard views for planners vs. technicians.

Limitations and Next Steps

This MVP is trained and demoed entirely on synthetic data, and its thresholds (Z_SCORE_LIMIT, CRITICAL_RUL, WARNING_RUL, HEALTHY_FRACTION) are hand-picked constants rather than values tuned against real failure records. Landing gear RUL accuracy is noticeably weaker than the other three component types in this run, likely due to fewer training examples for that type in a fleet this small. Before any production use, the models should be retrained and validated against real sensor/telemetry history and confirmed maintenance/failure events, and the MRO integration should follow the phased roadmap above.